

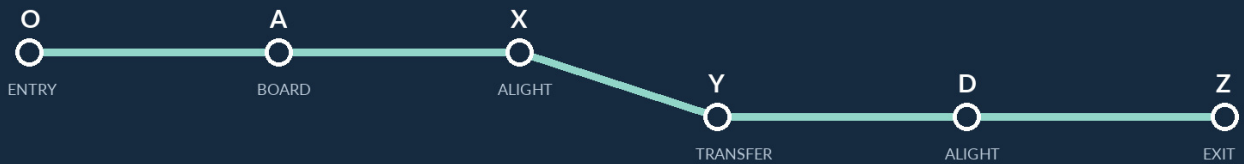
RAPTOR Reading Companion

An original guide for the EasySubway learning repository. Read this beside the research paper, not instead of it.

RAPTOR STUDY

From timetable events to trustworthy journeys.

Read the paper. Trace the rounds. Test the journey.



Start with one question

What can I reach with one more boarding? That question gives the round its meaning. Round zero admits walking; each subsequent round allows one additional vehicle. An earlier arrival and fewer transfers can disagree, so keep the useful tradeoff visible.

What this document is

This is original explanatory material, not a redistributed or translated copy of the RAPTOR paper. The runnable network uses invented stops, times, and hash-shaped identifiers. It is not an official passenger itinerary or an EasySubway release artifact.

Research source: Daniel Delling, Thomas Pajor, and Renato F. Werneck, *Round-Based Public Transit Routing*, ALENEX 2012. [Open the official publication page.](#)

01 / Protect the invariant

Read one round. Write the next. Keep the selected path witness stable.

Watch the labels improve.

A row means at most k boardings. Destination Z includes the exit walk.

BOARDINGS	O	A	X	Y	D	Z
0	08:00:00	08:01:00	unreached	unreached	unreached	unreached
1	08:00:00	08:01:00	08:10:00	08:12:00	08:29:00	08:30:00
2	08:00:00	08:01:00	08:10:00	08:12:00	08:21:00	08:22:00
3	08:00:00	08:01:00	08:10:00	08:12:00	08:21:00	08:22:00

08:22 with one transfer and 08:30 without a transfer are both nondominated.

RAPTOR STUDY / Original learning diagram / Synthetic examples

Say the label in plain English

A label at stop s in round k is the earliest known arrival using at most k boardings. Copying the previous round preserves already-feasible journeys. Boarding uses the previous-round arrival plus the admitted slack; it never reads a label written by another route in the same round.

Check the easy-to-miss details

Two boardings mean one transfer. Arrival at a platform is not yet arrival at the destination endpoint. Walking closure can require more than one edge relaxation. A non-overtaking trip order and compatible pickup/drop-off patterns justify this lab's single-active-trip scan.

Try a counterexample before an optimization: reverse the order of two connecting routes, add one second to a ready time, or supply a walking chain in reverse edge order. The answer should follow semantics, not iteration luck.

02 / Follow time and direction

Departure profiles, arrive-by, and last connection are different query problems.

Departure profiles

For a first-board departure d , access walk w , and boarding slack b , the latest catchable origin readiness is $d - w - b$. The lab processes those event thresholds latest to earliest and reuses round labels. Adjacent integer-second intervals merge only when their arrival/boarding signatures agree.

A minute-sampled loop is not the implemented range engine. An exhaustive second-by-second loop exists only in the independent test oracle. Walking-only profiles need nonconstant, affine arrival pieces; the restricted lab rejects that unsupported domain explicitly.

Reverse search

Start with the destination deadline. Use incoming indexes of existing forward walking edges, subtract durations, and scan trips backward. This reasons backward about a legal forward journey; it does not invent a reverse physical walkway.

Last connection

The final feasible journey follows the supplied service date's actual events. It is not defined by 23:59, midnight, or a convenient cutoff. The overnight fixture retains 24:06:00 as service seconds associated with its original date.

Ask a stronger question

Would these two intervals still be equivalent if the product promises a least-walking or safest-connection representative? The scalar lab cannot answer that full multicriteria question. That extension needs a richer intermediate frontier and a separate exact oracle.

Read repository chapters 03, 06, and 07. For the finished EasySubway target, follow Backend issues #305 through #310 and keep service-date, input-identity, and public-contract requirements separate.

03 / Connect the system

An algorithm result, an admitted artifact, and a deployed observation are not interchangeable.

Separate observed main from the target

The inspected EasySubway Backend main commit is **1d80b7afc58bf788dd76846ea7dc86fcb8f1cfaa**. Its algorithm ADR declares marked single-departure RAPTOR active and profile modes inactive pending PR #312. The study assumes relevant issues are implemented when describing the intended architecture; it does not claim that they are currently deployed.

Keep the owners clear

Data admits source evidence and immutable products. Backend is the routing authority. Mobile consumes generated contracts and presents accessible results, without a local routing fallback. Platform activates pinned runtime capabilities. Hub coordinates final evidence and identity decisions.

Prove the restricted model honestly

The independent oracle enumerates boardable trips and walking edges on a state graph. Point tests compare every stop and boarding budget; profile tests cover each integer second of small windows; reverse tests exhaust forward readiness. These are local correctness checks, not production performance or full Profile V1 acceptance evidence.

Read next

[Official RAPTOR publication](#)

[GTFS Schedule reference](#)

[EasySubway Backend #307: frontier and oracle target](#)

[EasySubway Backend #308: event-driven profile target](#)

Scope: one admitted service date, static walking, arrival/boarding routing, restricted range and last-connection domains, and an illustrative realtime overlay. Full ingestion, cryptographic admission, multicriteria routing, public endpoints, Mobile UI, and deployed activation are explained rather than implemented. Reviewed September 7, 2026.